

**МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ**

Федеральное государственное автономное образовательное учреждение
высшего профессионального образования

**НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ
ТОМСКИЙ ПОЛИТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ**

Инженерная школа информационных технологий и робототехники

Направление 15.03.06 «Мехатроника и робототехника»

Отделение автоматизации и робототехники

**Методы искусственного интеллекта. EDA. Линейная регрессия.
Дерево решений. CatBoost. XGBoost. Нейронные сети (MLP).**

Отчет по Лабораторной работе № 1

по дисциплине «Методы искусственного интеллекта в мехатронике и
робототехнике»

Вариант 22

Выполнил:

студент группы 8ЕМ51

Подпись

Дата

Степанов А.С.

Фамилия И.О.

Проверил:

Старший преподаватель
(ОАР, ИШИТР)

Должность

Подпись

Дата

Павловский А.В.

Фамилия И.О.

Оглавление

ЦЕЛЬ РАБОТЫ.....	3
ЗАДАНИЯ.....	3
1 ОПИСАНИЕ ДАТАСЕТА И ОПРЕДЕЛЕНИЕ ПРИЗНАКОВ	4
2 ПОСТРОЕНИЕ ПАЙПЛАЙНА (DVC).....	9
3 РЕАЛИЗАЦИЯ ЛИНЕЙНОЙ РЕГРЕССИИ.....	11
4 РЕАЛИЗАЦИЯ ДЕРЕВЬЕВ РЕШЕНИЙ	12
5 РЕАЛИЗАЦИЯ CATBOOST.....	14
6 РЕАЛИЗАЦИЯ XGBOOST	16
7 РЕАЛИЗАЦИЯ НЕЙРОННОЙ СЕТИ.....	18
8 КОНЕЧНЫЙ ВЫЧИСЛИТЕЛЬНЫЙ ГРАФ	21
9 ВЫВОД ПО ИТОГОВОЙ ТАБЛИЦЕ С МЕТРИКАМИ.....	22

ЦЕЛЬ РАБОТЫ

Получить навыки анализа первичных данных и определения признаков взаимосвязи (EDA), понимания моделей: линейная регрессия, дерево решений, CatBoost, XGBoost, нейронные сети (MLP) и умения разрабатывать программу на языке Python для реализации представленных моделей.

ЗАДАНИЯ

Для выполнения лабораторной работы необходимо выполнить следующие пункты:

1. Описать датасета и определить влияние признаков и выбрать признаки, которые наиболее подходят для поставленной задачи предсказания (EDA);
2. Построить пайплайн (DVC) исходя из результатов EDA;
3. Реализовать линейную регрессию, определить веса, метрики и ошибки;
4. Реализовать дерево решений, определить метрики и ошибки. Привести рисунок первых узлов дерева решений;
5. Реализовать CatBoost, определить метрики и ошибки. Выгрузить Feature Importance;
6. Реализовать XGBoost, определить метрики и ошибки. Выгрузить Feature Importance;
7. Реализовать нейронную сети, определить метрики, ошибки, кривые обучения, гистограммы весов с интерпретацией и график из Tensorboard;
8. Выгрузить конечный вычислительный граф DVC;
9. Выгрузить конечный вычислительный граф DVC;
10. Сделать вывод по работе.

1 ОПИСАНИЕ ДАТАСЕТА И ОПРЕДЕЛЕНИЕ ПРИЗНАКОВ

Данные были собраны Ноа (<https://data.world/nrippner/ols-regression-challenge>) из многочисленных источников, а именно из American Community Survey ([census.gov](https://www.census.gov)), [clinicaltrials.gov](https://www.clinicaltrials.gov) и [cancer.gov](https://www.cancer.gov).

Основные признаки, содержащиеся в датасете:
TARGET_deathRate: Зависимая переменная. Среднее значение смертности от рака на душу населения (100 000)(a);

avgAnnCount: Среднее число зарегистрированных случаев рака, диагностируемых ежегодно(a);

avgDeathsPerYear: Среднее число зарегистрированных случаев смерти от рака(a);

incidenceRate: Среднее число диагнозов рака на душу населения (100 000)(a);

medianIncome: Медианный доход по округу (b);

popEst2015: Население графства (b);

povertyPercent: Процент населения, живущего в бедности (b);

studyPerCap: Число клинических исследований, связанных с раком, на душу населения в каждом округе (a);

binnedInc: Медианный доход на душу населения в разбивке по децилям (b);

MedianAge: средний возраст жителей округа (b);

MedianAgeMale: Средний возраст мужчин, проживающих в округе (b);

MedianAgeFemale: Средний возраст женщин, проживающих в округе (b);

География: Название округа (b);

AvgHouseholdSize: Средний размер домохозяйства в округе (b);

PercentMarried: Процент жителей округа, состоящих в браке (b);

PctNoHS18_24: Процент жителей округа в возрасте 18-24 лет с высшим образованием: менее средней школы (b);

PctHS18_24: Процент жителей округа в возрасте 18-24 лет, получивших высшее образование: диплом средней школы (b);

PctSomeCol18_24: Процент жителей округа в возрасте 18-24 лет, получивших высшее образование: некоторый колледж (b);

PctBachDeg18_24: Процент жителей округа в возрасте 18-24 лет, получивших высшее образование: степень бакалавра (b);

PctHS25_Over: Процент жителей округа в возрасте 25 лет и старше, получивших высшее образование: диплом средней школы (b);

PctBachDeg25_Over: Процент жителей округа в возрасте 25 лет и старше, получивших высшее образование: степень бакалавра (b);

PctEmployed16_Over: Процент жителей округа в возрасте 16 лет и старше, имеющих работу (b);

PctUnemployed16_Over: Процент жителей округа в возрасте 16 лет и старше, не имеющих работы (b);

PctPrivateCoverage: Процент жителей округа с частным медицинским страхованием (b);

PctPrivateCoverageAlone: Процент жителей округа, имеющих только частную медицинскую страховку (без государственной помощи) (b);

PctEmpPrivCoverage: Процент жителей округа с частной медицинской страховкой, предоставляемой работником (b);

PctPublicCoverage: Процент жителей округа с государственным медицинским страхованием (b);

PctPublicCoverageAlone: Процент жителей округа, имеющих только государственную медицинскую страховку (b);

PctWhite: Процент жителей округа, идентифицирующих себя как белые (b);

PctBlack: Процент жителей округа, идентифицирующих себя как чернокожих (b)

PctAsian: Процент жителей округа, идентифицирующих себя как азиатов (b);

PctOtherRace: процент жителей округа, которые относят себя к категории, не являющейся белой, черной или азиатской (b);

PctMarriedHouseholds: Процент домохозяйств, состоящих в браке (b);

BirthRate: Количество живорождений по отношению к количеству женщин в округе (b).

В файле “cancer_reg.csv” представлена первоначальная предобработка данных, проходившая по следующим этапам.

С помощью команды “df.shape” была проанализирована размерность файла (количество строк и столбцов).

С помощью команды “df.head()” были выведены первые 5 строк.

С помощью команды “df.info()” был проанализирован каждый признак на наличие размерности значений и количества не нулевых ячеек в таблице.

С помощью команды “df.describe()” были простотрены основные статистические данные, такие как, количество значений, среднее значение, минимальное и максимальные значения.

Следующим шагом стала замена пропущенных (нулевых) значений медианным значением столбца. После датасет был ещё раз проверен на количество пропущенных значений, число которых оказалось равно нулю и построена гистограмма распределения смертности населения на 100000 человек, представленная на рисунке 1.

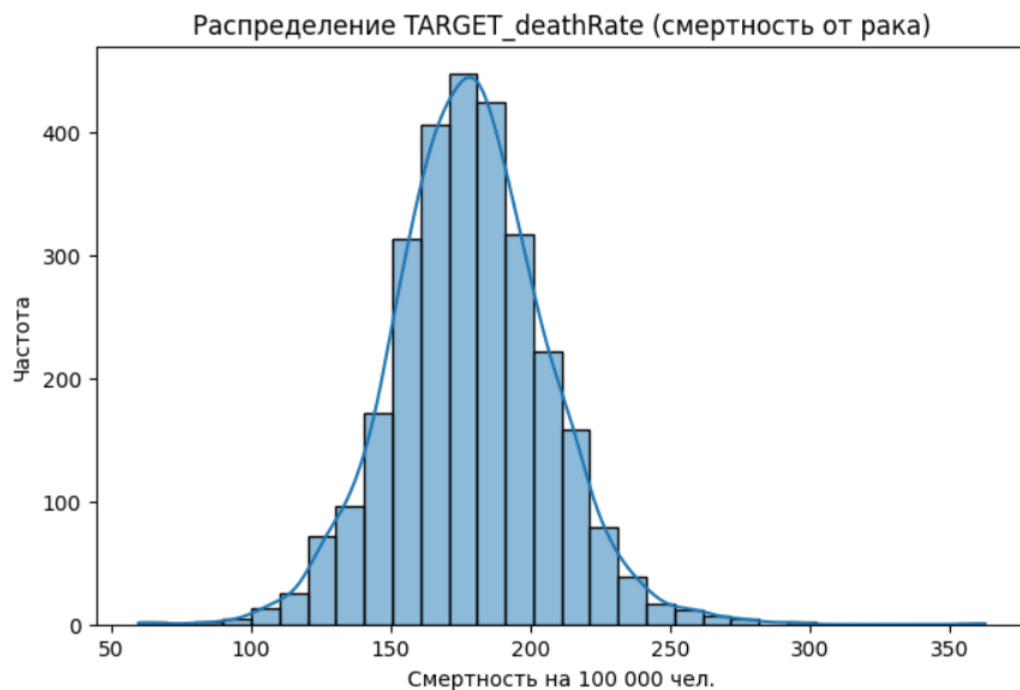


Рисунок 1 – Диаграмма смертности населения в зависимости от частоты.

Затем был построен график корреляции признаков от целевого значения, представленный на рисунке 2.

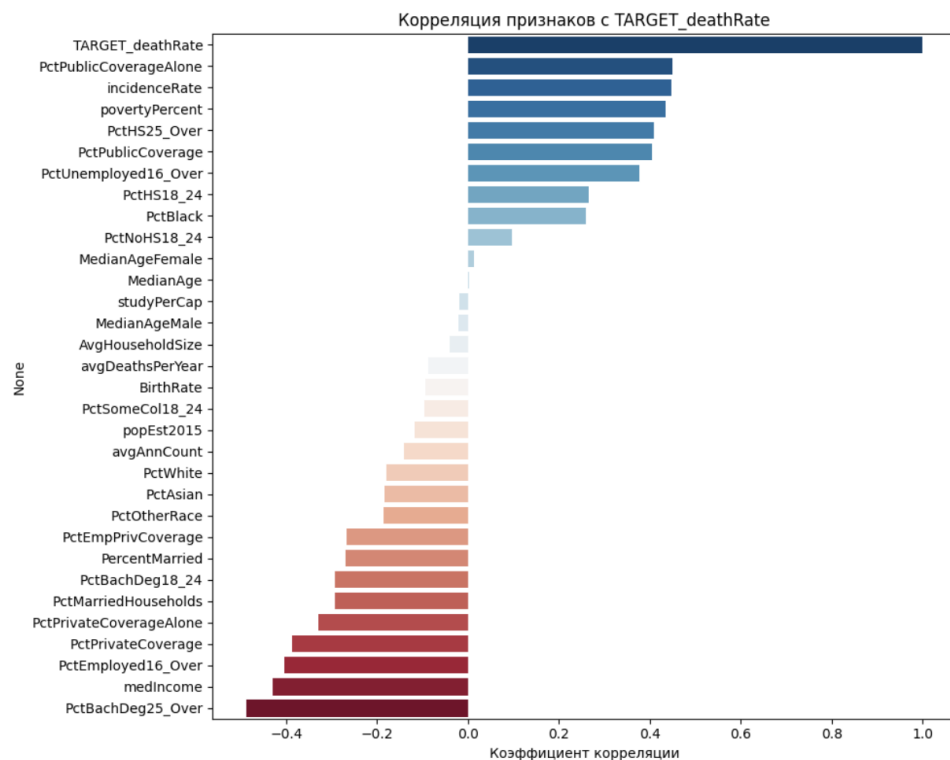


Рисунок 2 – График корреляционной зависимости от целевого значения.

В результате чего были отобраны признаки, имеющие значение выше 0. По результатам отобранных признаков была построена корреляционная

матрица, показывающая зависимость признаков друг от друга и от целевого значения, представленная на рисунке 3.

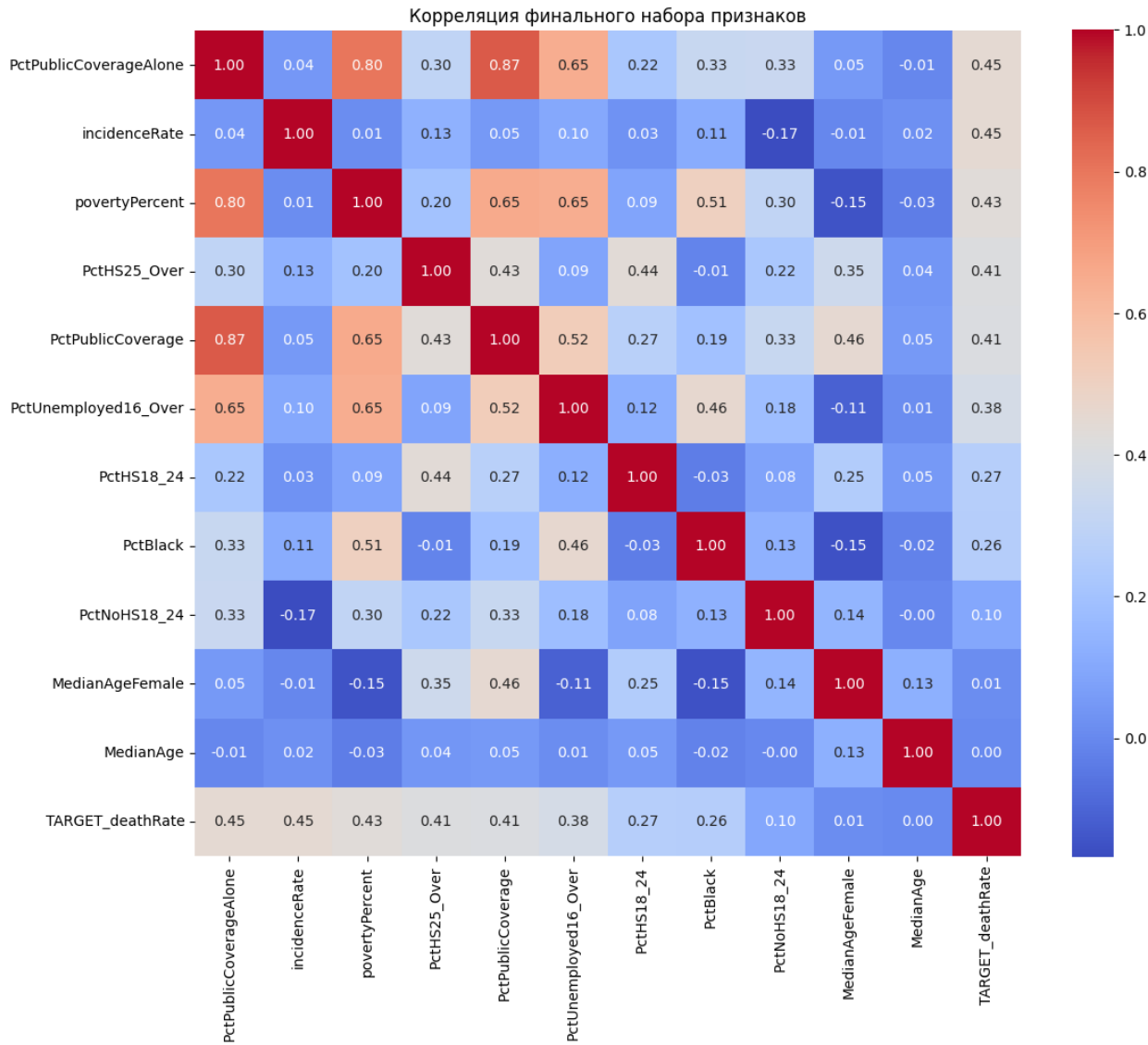


Рисунок 3 – Тепловая карта корреляций между потенциальными признаками

По приведенным результатам был сформирован новый датасет, для дальнейшей работы.

2 ПОСТРОЕНИЕ ПАЙПЛАЙНА (DVC)

Следующей задачей стало построение пайплайна в контексте DVC. Это необходимо для структурирования последовательности этапов, определяющих рабочий процесс и зависимости для задач машинного обучения.

Основные функции DVC в контексте разрабатываемой задачи:

1. Систематизация сценариев, т.е. выделение отдельных скриптов для загрузки, обработки данных, генерации признаков, обучения модели;
2. Запуск всех частей пайплайна одной командой;
3. Перезапуск стадий при изменении зависимостей или параметров;
4. Кэширование запусков и их результатов для избежания ненужных перезапусков.

Структура DVC выглядит следующим образом:

Листинг 1 – Структура DVC модели

```
stages:

  prepare: #Загрузка подготовленных данных
    cmd: #Запуск файла для подготовки данных
    deps:
    outs: #Создаются файлы для обучения моделей
  model_train:
    cmd: #Запуск обучающей модели
    deps: #Загрузка данных для обучения, запуск программы с
основными функциями, загрузка параметров для подбора лучшей модели
    outs: #Сохранение гистограмм весов и кривых обучения
    metrics: #Сохранение выходных метрик
  model_validate:
    cmd:
    deps:
    outs:
    metrics:
```

```
model_test:  
  cmd:  
  deps:  
  outs:  
  metrics:
```

Данная DVC структура прописана для всех используемых моделей.

3 РЕАЛИЗАЦИЯ ЛИНЕЙНОЙ РЕГРЕССИИ

Для реализации линейной регрессии была использована библиотека `sklearn` из которой была позаимствована функция `LinearRegression`. Из этой же библиотеки была позаимствована функция `StandardScaler` для стандартизации признаков. На рисунке 4 представлен график зависимости предсказанных значений от действительных. В таблице 1 представлены метрики обучения линейной регрессии.

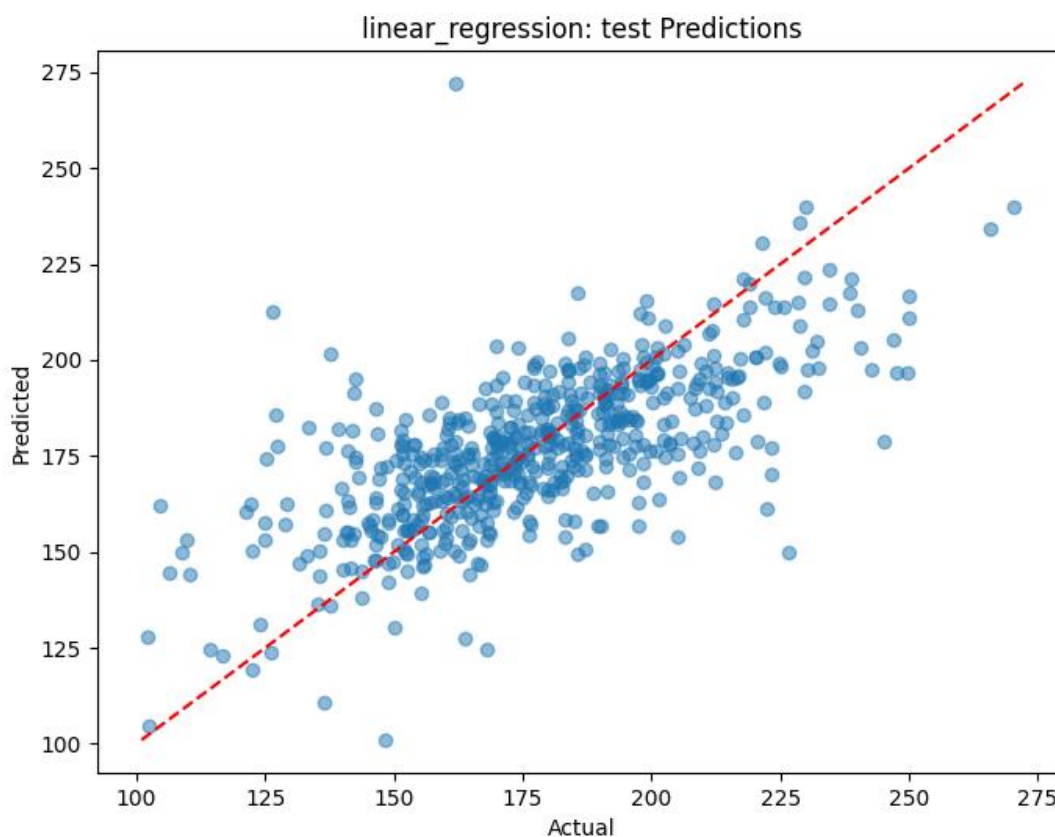


Рисунок 4 – График зависимости предсказанных значений от действительных

Таблица 1 – Метрики обучения линейной регрессии

	Значения			
Метрики	MSE	MAE	RMSE	R^2
Train	346.344	13.879	18.61	0.536
Validate	394.024	14.609	19.85	0.522
Test	399.961	14.577	19.999	0.467

Метрики на train, validate и test близки между собой – переобучения нет. R^2 около 0.47–0.54 означает, что линейная регрессия объясняет примерно половину вариации смертности. Ошибки (MAE ~14.5) говорят о том, что модель ошибается в среднем на 14–15 случаев смерти на 100 тыс. человек.

4 РЕАЛИЗАЦИЯ ДЕРЕВЬЕВ РЕШЕНИЙ

Для реализации линейной регрессии была использована библиотека sklearn из которой была позаимствована функция DecisionTreeRegressor. Из этой же библиотеки была позаимствована функция plot_treed для построения графика узлов деревьев решений. На рисунке 5 представлен график зависимости предсказанных значений от действительных. На рисунке 6 представлены первые узлы дерева решений. В таблице 2 представлены метрики обучения линейной регрессии.

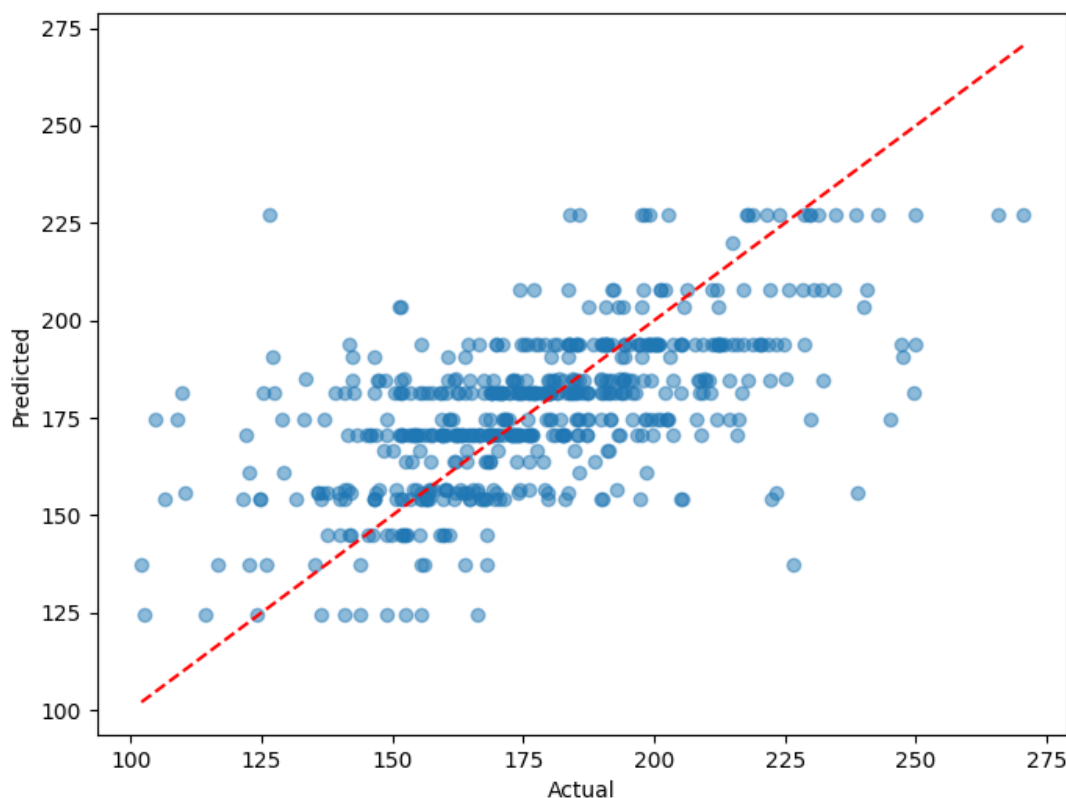


Рисунок 5 – График зависимости предсказанных значений от действительных

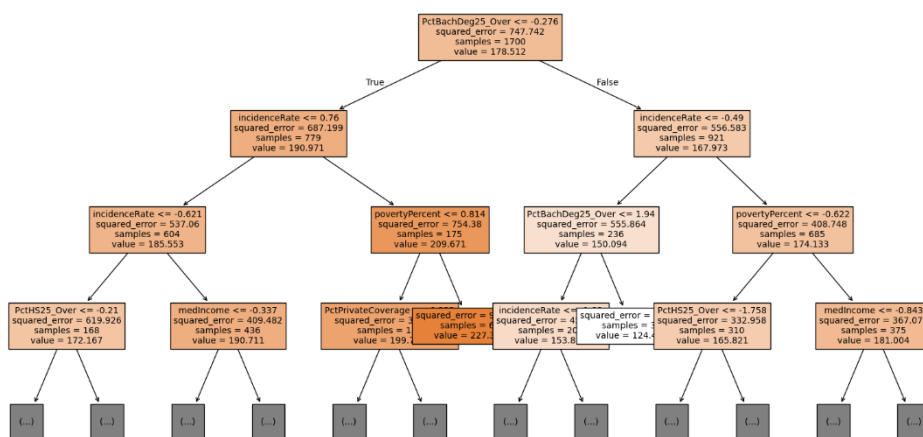


Рисунок 6 - Первые узлы дерева решений

Таблица 2 – Метрики обучения дерева решений

	Значения			
Метрики	MSE	MAE	RMSE	R ²
Train	354.928	14.289	18.839	0.525
Validate	488.106	17.02	22.093	0.407
Test	489.669	16.318	22.128	0.347

R² на тесте 0.347 - хуже, чем у линейной регрессии (0.467). Наблюдается небольшое переобучение: train (0.525) > test (0.347). Ошибки MAE выше (~16.3 против 14.6). Дерево решений без настройки параметров показало более слабый результат, чем линейная модель.

5 РЕАЛИЗАЦИЯ CATBOOST

Для реализации catboost была использована библиотека catboost из которой была позаимствована функция CatBoostRegressor. Из библиотеки matplotlib была позаимствована функция plt для построения графиков предсказанных значений. На рисунке 7 представлен график зависимости предсказанных значений от действительных. На рисунке 8 зависимости признаков от их важности для предсказания. В таблице 3 представлены метрики обучения CatBoost.

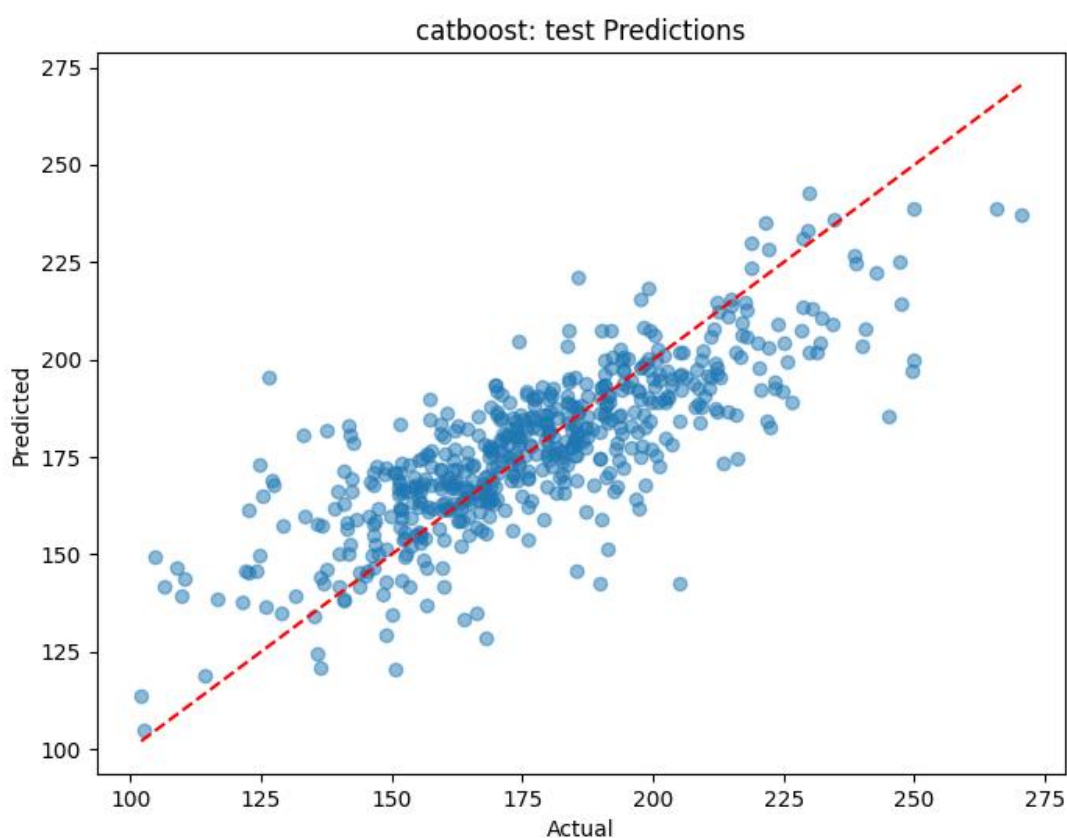


Рисунок 7 – График зависимости предсказанных значений от действительных

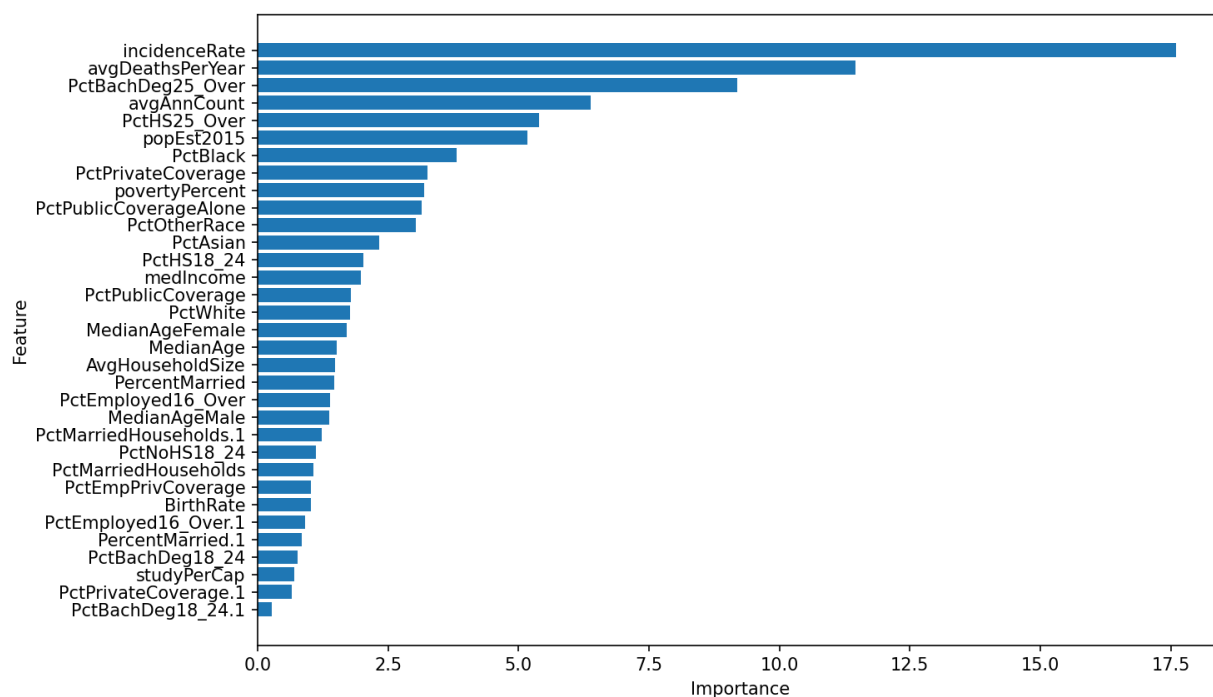


Рисунок 8 – График зависимости признаков от их важности для предсказания

Таблица 3 – Метрики обучения CatBoost

Метрики	Значения			
	MSE	MAE	RMSE	R ²
Train	27.754	4.128	5.268	0.962
Validate	264.745	11.699	16.271	0.678
Test	275.884	12.419	16.609	0.632

Очень сильное переобучение на тренировочных данных $R^2 = 0.962$ (почти идеально), а на тесте – 0.632. CatBoost выучил тренировочные данные почти наизусть, но обобщает хуже. Тем не менее, результат на тесте ($R^2=0.632$) лучше, чем у линейной регрессии и дерева. MAE ~12.4 – ошибка меньше, чем у предыдущих моделей. Это типичное поведение градиентного бустинга при отсутствии регуляризации или при малом объёме данных.

6 РЕАЛИЗАЦИЯ XGBOOST

Для реализации xgboost была использована библиотека xgboost из которой была позаимствована функция XGBRegressor. Из библиотеки matplotlib была позаимствована функция plt для построения графиков предсказанных значений. На рисунке 7 представлен график зависимости предсказанных значений от действительных. На рисунке 8 зависимости признаков от их важности для предсказания. В таблице 4 представлены метрики обучения CatBoost.

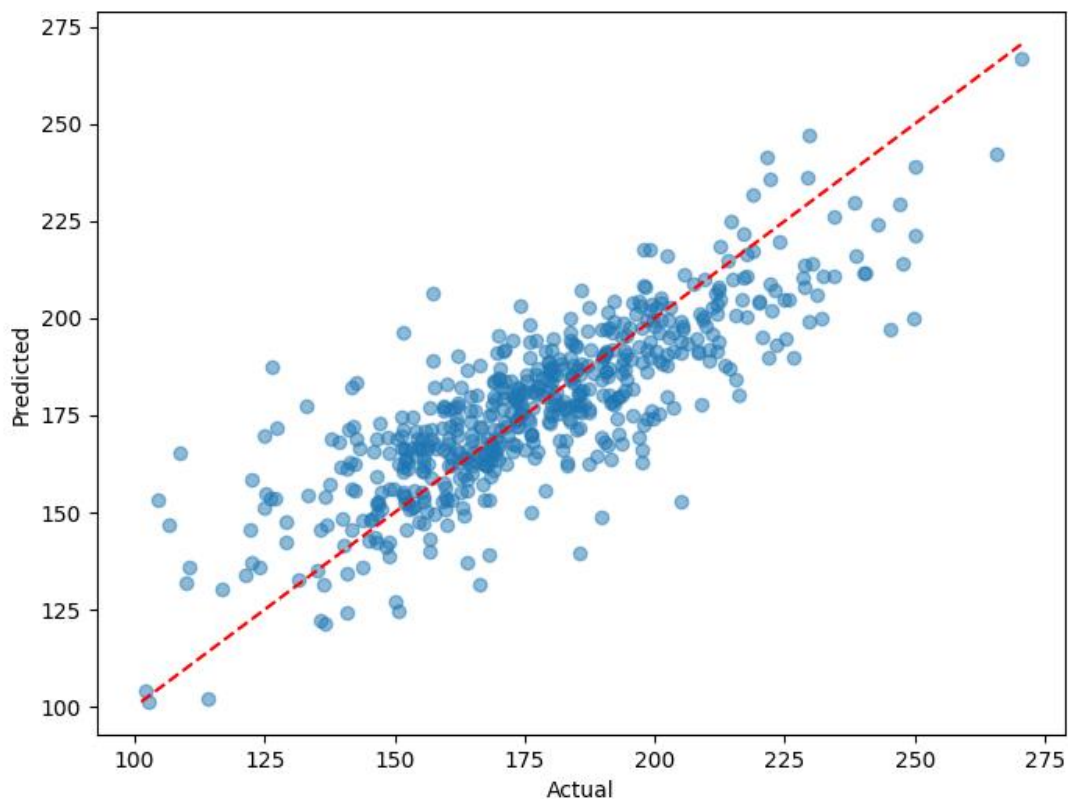


Рисунок 9 – График зависимости предсказанных значений от действительных

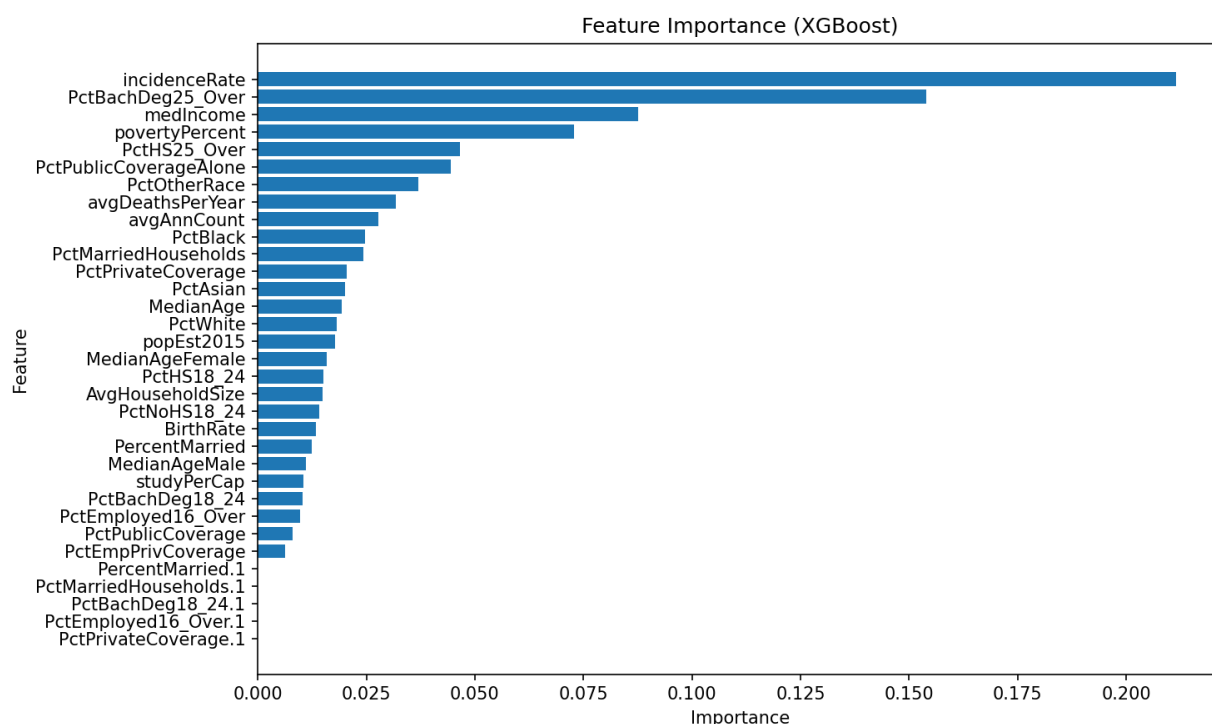


Рисунок 10 – График зависимости признаков от их важности для предсказания

Таблица 4 – Метрики обучения XGBoost

Метрики	Значения			
	MSE	MAE	RMSE	R ²
Train	18.193	3.283	4.265	0.975
Validate	238.694	11.403	15.449	0.710
Test	244.255	11.819	15.628	0.674

Аналогично CatBoost: сильное переобучение (R^2 train 0.975 против 0.674 на тесте). На тесте XGBoost показывает наилучший $R^2 = 0.674$ среди всех моделей. MAE = 11.82 – самая низкая ошибка в абсолютных значениях. XGBoost оказался лучшим по качеству предсказания на тестовых данных.

7 РЕАЛИЗАЦИЯ НЕЙРОННОЙ СЕТИ

Для реализации нейронной сети была использована библиотека `sklearn` из которой была позаимствована функция `KerasRegressor`. Из библиотеки `matplotlib` была позаимствована функция `plt` для построения графиков предсказанных значений. На рисунке 11 представлены графики кривых обучения. На рисунке 12 представлен график расположения предсказанных значений относительно актуальных. На рисунке 13 изображены графики гистограмм весов для каждого из слоев, на которых видно, что Веса сосредоточены около нуля, распределение симметрично - градиенты не взрываются. Смещения также имеют небольшой разброс, модель не смещена. Отсутствуют аномальные выбросы - обучение прошло стабильно.

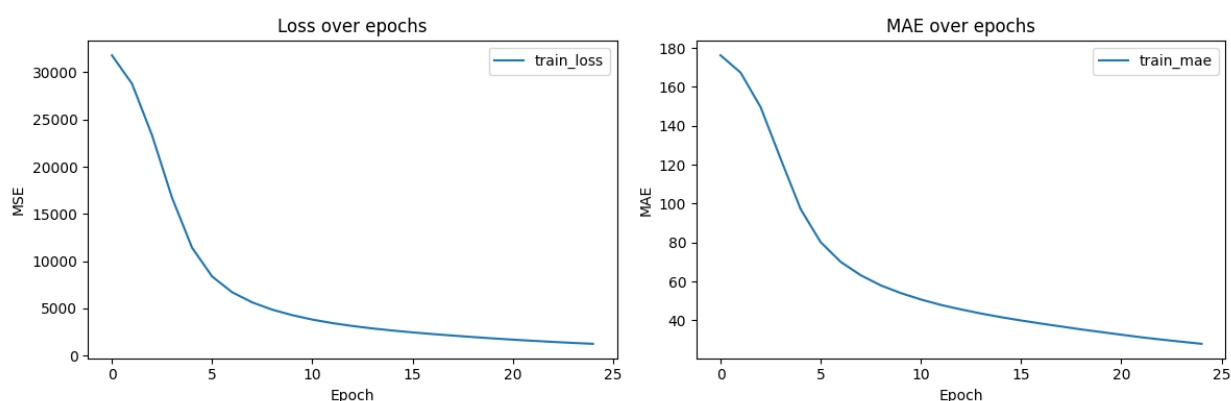


Рисунок 11 – График кривых потерь при обучении

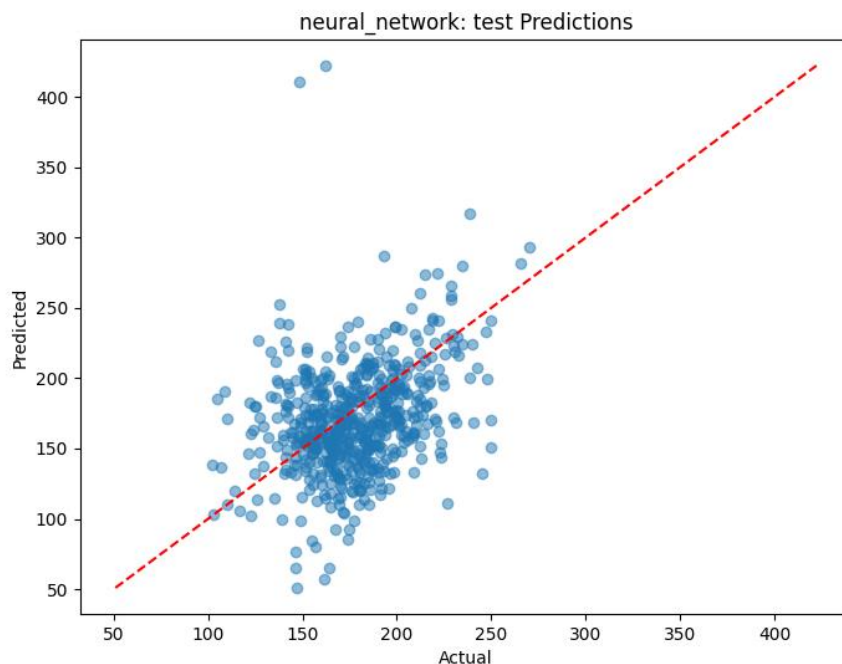


Рисунок 12 – График расположения предсказанных значений относительно актуальных

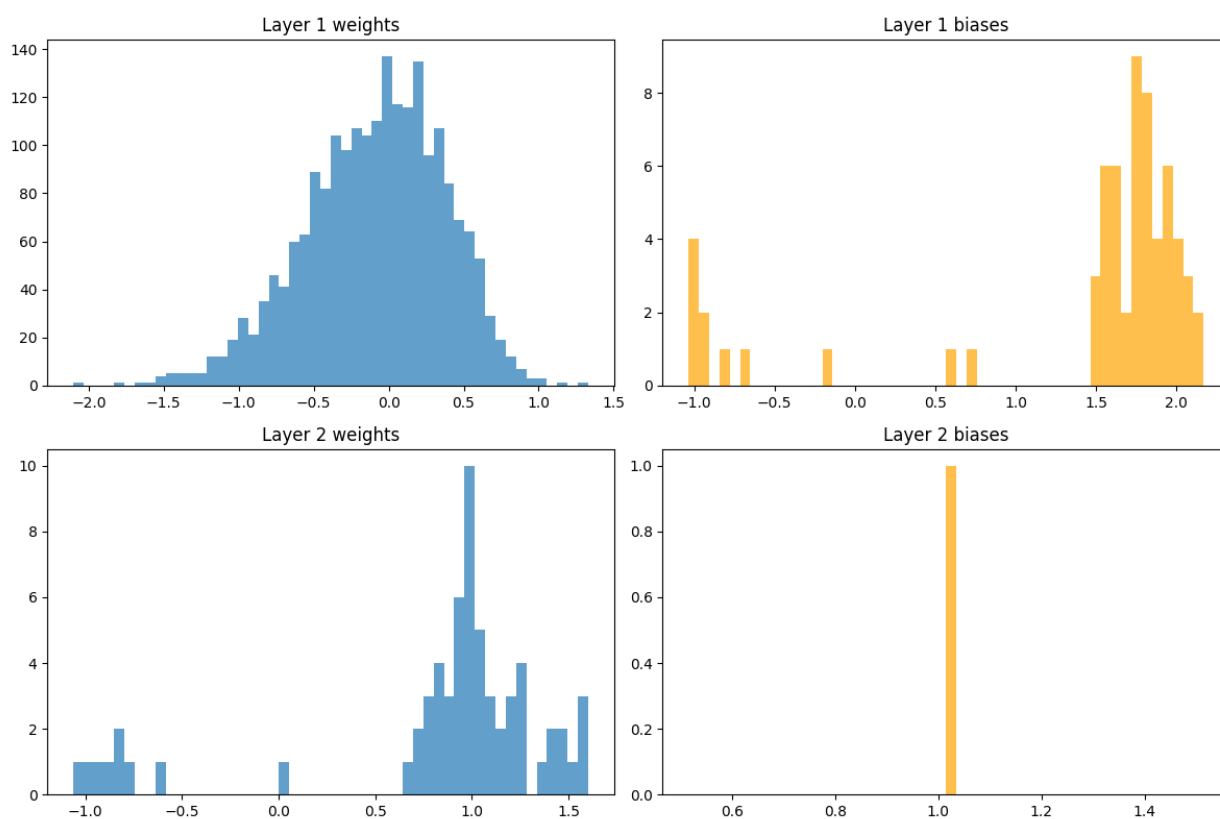


Рисунок 13 – Графики гистограмм весов для каждого из слоев

Таблица 5 - Метрики обучения MLP

Метрики	Значения			
	MSE	MAE	RMSE	R^2
Train	318.930	13.638	17.858	0.573
Validate	515.723	16.46	22.709	0.374
Test	506.421	16.652	22.503	0.324

R^2 на тесте 0.324 – самый низкий среди всех моделей. MLP справилась хуже всего. Есть заметное переобучение (train 0.573, test 0.324). Ошибки MAE ~16.65 – выше, чем у линейной регрессии и бустингов. Возможные причины связаны с недостатком данных, неудачной функцией активации или скоростью обучения.

8 КОНЕЧНЫЙ ВЫЧИСЛИТЕЛЬНЫЙ ГРАФ

При построении пайплайна была также построен и выгружен вычислительный граф DVC, изображенный на рисунке 14.



Рисунок 14 - Вычислительный граф DVC

9 ВЫВОД ПО ИТОГОВОЙ ТАБЛИЦЕ С МЕТРИКАМИ

Для объективной оценки обобщающей способности каждой модели использовались метрики, полученные на тестовой выборке, которая не участвовала ни в обучении, ни в настройке гиперпараметров. В таблице 19 приведены финальные показатели всех пяти реализованных алгоритмов.

Таблица 6 – Сравнение метрик моделей на тестовой выборке

	Значения			
Метрики	MSE	MAE	RMSE	R ²
Test LR	399.961	14.577	19.999	0.467
Test DT	489.669	16.318	22.128	0.347
Test CAT	275.884	12.419	16.609	0.632
Test XG	244.255	11.819	15.628	0.674
Test MLP	506.421	16.652	22.503	0.324

Лучшая модель по всем метрикам – XGBoost ($R^2 = 0.674$, MAE = 11.82). Вторая – CatBoost ($R^2 = 0.632$). Линейная регрессия дала средний результат ($R^2 = 0.467$). Дерево решений и нейронная сеть показали худшее качество ($R^2 < 0.35$). Все модели страдают от некоторого переобучения. Для улучшения MLP потребовалась бы более тщательная настройка гиперпараметров и увеличение объёма данных.